

피카츄 배구 봇 대회 전략 · 구현 · 튜닝 가이드

공개된 대회 엔진 코드 기준 / PikaPlanner v1

목표: “공을 따라가는 봇”이 아니라, 다음 접촉을 설계하고
상대가 받기 어려운 궤적을 만드는 모델 기반 봇

2026-09-01

0. 이 문서를 어떻게 읽으면 되는가

이 문서는 대회 코드를 직접 수정할 팀원이 “왜 이런 구조인지”까지 이해하도록 만든 실전용 설명서입니다. 먼저 1~4 장을 읽으면 게임과 전략의 핵심을 잡을 수 있고, 실제 개발 때는 7~10 장을 옆에 두고 쓰면 됩니다.

가장 중요한 한 문장

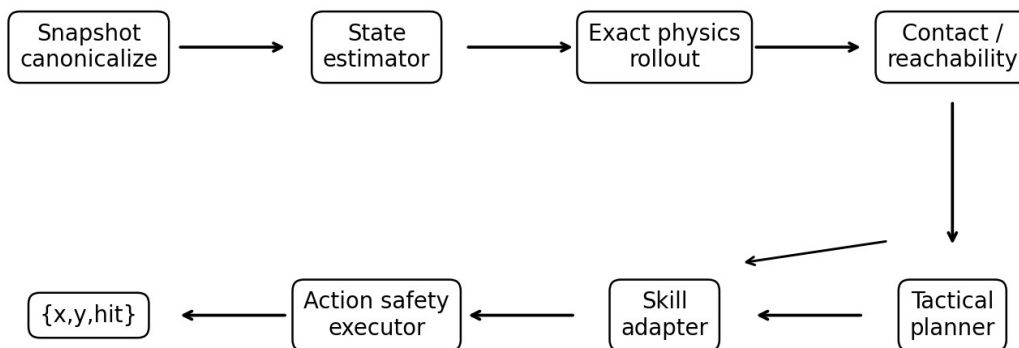
이 게임에서 진짜 중요한 것은 예상 낙하지점 자체가 아니라 “다음 공-플레이어 접촉을 언제, 어디서, 어떤 상태로 만들 것인가”입니다.

파일	역할
PikaPlanner_tuned_v1.js	현재 추천 제출 후보. 오프라인 튜닝값이 삽입된 버전
PikaPlanner_v1.js	개발 원본. 스킬 adapter 와 파라미터 구조를 보기 좋게 유지
tools/sim_engine.mjs	공개된 물리와 bot tick 을 옮긴 독립형 근사 시뮬레이터
tools/tune.mjs	여러 파라미터 조합을 자동 대전으로 비교
SKILL_DAY_CHECKLIST.md	스킬 공개 당일 수정 순서 체크리스트

1. 최종 결론: 이 게임은 어떤 문제인가

실전적으로는 “작은 정확 물리 모델을 거의 완전히 알고 있는 2 인 적대적 제어 문제”로 보는 것이 가장 좋습니다. 매 3 프레임마다 관측하지만 행동 결과는 Worker 응답이 도착한 다음 프레임부터 적용될 수 있으므로, 고정 120ms 지연이 아니라 40ms 단위로 흔들리는 비동기 sampled-data control 에 가깝습니다.

- 공/플레이어 물리는 공개되어 있으므로 RL 에게 물리를 다시 학습시킬 이유가 작습니다.
- expectedLandingPointX 는 훌륭한 수비 shortcut 이지만, 강한 공격/수비의 중심 변수는 contact 입니다.
- 점프·다이빙은 되돌리기 어려운 commit 행동이고, 수평 이동은 매 프레임 방향을 바꿀 수 있는 값싼 행동입니다.
- 따라서 “언제 commit 할지”와 “어떤 접촉 window 를 만들지”가 승부를 크게 가릅니다.



Core planner stays stable; tournament-day skill changes are isolated in the adapter.

그림 1. PikaPlanner 의 전체 구조

2. 공개 엔진에서 확정된 핵심 물리

항목	정확한 값/규칙	전략적 의미
수평 이동	6 px/frame	한 bot 관측 주기(3 frame) 동안 최대 약 18px 이동
다이빙	8 px/frame, 시작 $v_y = -5$	긴급 reachable set 확장. 이후 누워서 회복 비용 발생
점프	시작 $v_y = -16$, 매 frame +1 중력	점프 궤적을 정수 단위로 정확히 계산 가능
플레이어 충돌	$ ball.x - player.x \leq 32, ball.y - player.y \leq 32$	원형이 아니라 64x64 정사각형 hitbox
일반 몸타격	$ v_x = \text{floor}(dx /3), v_y = -\max(v_y , 15)$	공의 어느 면을 맞히는지가 방향/속도를 결정
Power x	$x=0 \rightarrow v_x =10, x!=0 \rightarrow v_x =20$	파워히트는 느림/빠름 두 수평 모드
Power y	$v_y = 2 * \text{기존 } v_y * y_{\text{Input}}$	같은 DOWN 도 접촉 시점에 따라 강도가 달라짐
벽	공 중심 x가 0/432 밖으로 갈 때 반사	공이 화면 밖으로 반쯤 나가는 대칭 버전
네트	$x=216 \pm 25, y$ 조건별 top/side 반사	경계가 불연속이지만 결정론적

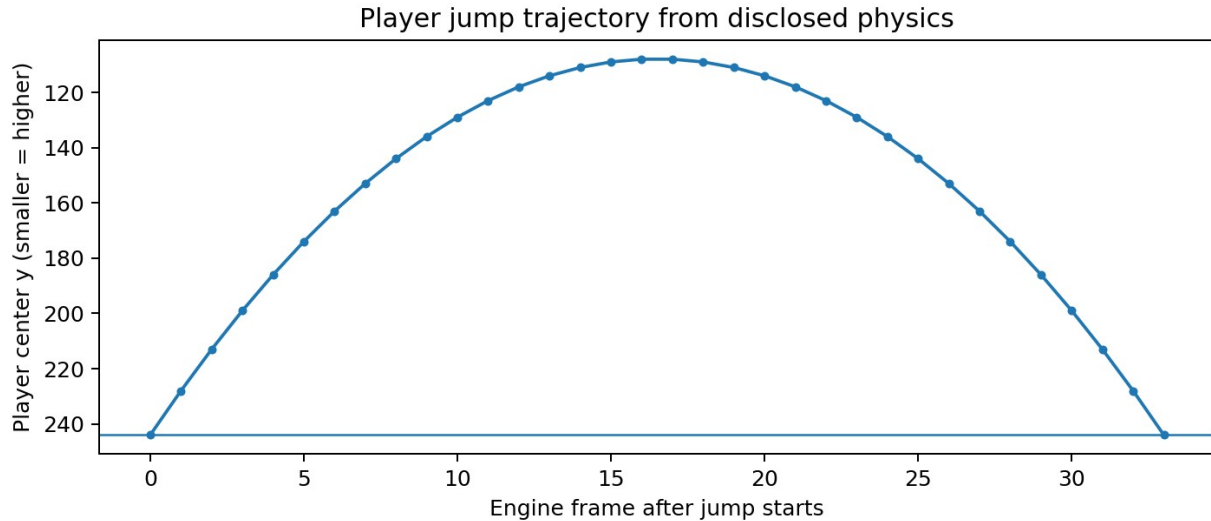


그림 2. 공개된 점프 식으로 계산한 플레이어 중심 y 궤적

최고점은 대략 $y=108$ 부근입니다. 플레이어 hitbox가 위아래 32이므로 공 중심이 약 $y=76$ 까지 내려오면 접촉 가능한 구간이 생깁니다. 이 때문에 “공이 높으니 점프” 같은 규칙보다, 미래 공 궤적과 점프 궤적이 실제로 겹치는 프레임을 찾는 방식이 정확합니다.

3. 가장 헷갈리는 부분: bot tick 과 실제 입력 지연

botInput.js의 실제 실행 순서를 보면 getInput()은 매 게임 프레임 실행됩니다. 다만 snapshot 요청은 $\text{tick} \% 3 == 0$ 일 때만 Worker로 보내고, 그 프레임의 physics에는 이미 존재하던 latestAction이 들어갑니다. 새 결과가 빨리 돌아오면 다음 engine frame부터 적용될 수 있습니다.

```

frame 3: snapshot 생성 -> 이전 action으로 physics
        Worker에서 decide(snapshot) 계산
frame 4: 결과가 도착했다면 새 action 적용 가능
frame 5: 같은 action hold
frame 6: 다음 snapshot 생성 ...

```

중요한 정정

문서의 “항상 약 1 tick(120ms) 지연”은 구현을 단순화한 설명입니다. 실제로는 관측 주기가 120ms 이고, 빠른 JS Worker 라면 결과가 그 다음 40ms 프레임부터 적용될 수 있습니다. 그래서 우리 실험기는 1/2/3-frame latency 를 섞어 테스트합니다.

입력은 pulse 가 아니라 hold 다

반환한 {x,y,hit}는 다음 Worker 결과가 들어오기 전까지 계속 유지됩니다. 이것이 생각보다 위험합니다. 예를 들어 점프 중 hit=1, x!=0 이 남아 있는데 착지하면서 state 가 0 이 되면, 같은 프레임/다음 프레임에 dive 조건으로 바뀔 수 있습니다. y=-1 도 착지 뒤까지 남으면 재점프를 만들 수 있습니다.

- 그래서 planner 가 만든 “의도”를 곧바로 반환하지 않습니다.
- 마지막 Action Safety Executor 가 state 와 예상 착지 시점을 보고 hit/y 를 다시 gate 합니다.
- 대회 당일 스킬 입력도 반드시 이 safety layer 를 통과시키는 편이 좋습니다.

4. 전략을 아주 쉽게 이해하기

4.1 공이 어디 떨어지는지보다, 어디에서 맞을지를 먼저 본다

예상 낙하지점은 아무도 공을 다시 건드리지 않았을 때의 마지막 결과입니다. 좋은 봇은 그 전에 공을 만집니다. 그러므로 “낙하지점으로 이동”은 보험이고, 공격적으로는 미래의 접촉 후보를 계산해야 합니다.

현재 상태

- > 공을 1,2,3,... frame 앞으로 시뮬레이션
- > 그때 내 몸/점프 궤적으로 닿을 수 있는가?
- > 닿는 후보들 중 공격하기 좋은 접촉은 무엇인가?

4.2 몸으로 받을 때도 위치가 공격 방향을 만든다

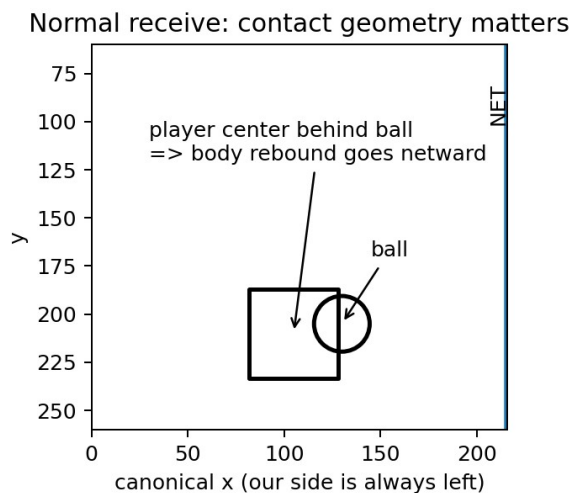


그림 3. 공 바로 밑보다 “공보다 뒤쪽”에서 받는 이유

일반 collision 은 공과 플레이어 중심의 x 차이로 공의 vx 를 만듭니다. 공 중심과 정확히 겹치면 vx=0 이 되어 랜덤 분기까지 생깁니다. 그래서 PikaPlanner 는 일반 리시브 때 일부터 9~30px 정도 뒤에 서서 “의도된 방향”을 만듭니다.

4.3 깊은 공은 한 번 self-set 할 가치가 있다

처음 버전은 닿는 공마다 바로 상대 코트로 보냈습니다. 자체 실험에서 몸으로 끈질기게 받아내는 수비형과 장기 랠리에서 불리했습니다. 그래서 깊은 공은 한 번 작은 offset 으로 받아 공을 자기 코트의 네트 근처에 띄우고, 다음 접촉에서 점프 파워히트를 노리는 선택지를 추가했습니다.

5 터치와의 관계

self-set 은 무한히 반복하지 않습니다. 현재 붓은 한 possession 에서 공격 구축용 self-set 을 사실상 한 번만 사용하도록 보수적으로 설계했습니다. 5 번째 접촉 실패 직전까지 욕심내는 구조가 아닙니다.

5. 공격의 중심: Power Hit rollout

공중에서 공격 가능한 상태가 되면 버튼을 감으로 고르지 않습니다. 후보를 실제 physics 로 굴러보고 자기 코트에 박히는 공격을 버린 뒤, 상대가 시간 안에 도달하기 어려운 결과를 선택합니다.

가족	x 입력	y 입력	특징
SLOW UP	0	-1	느린 아치
FAST UP	± 1	-1	빠른 아치
SLOW FLAT	0	0	느린 직선
FAST FLAT	± 1	0	빠른 직선
SLOW DOWN	0	+1	느린 하방
FAST DOWN	± 1	+1	빠른 하방

특히 DOWN 의 수직속도는 “2 × 접촉 직전 |ball.vy|”입니다. 따라서 공이 빨리 떨어지는 순간의 DOWN 은 같은 버튼이어도 훨씬 강합니다. 반대로 네트에서 너무 멀면 자기 코트에 먼저 꽂힐 수 있으므로 exact rollout 으로 제거합니다.

Shot 평가가 보는 것

- 실제로 상대 코트에 착지하는가?
- 상대 현재 x 에서 착지점까지 필요한 거리와 남은 시간이 얼마인가?
- 상대가 dive/lying 처럼 이미 commit 한 상태인가?
- 착지까지 시간이 짧아 반응 여유가 적은가?
- 코트 끝/짧은 공처럼 상대의 이동 폭을 크게 요구하는가?

6. 수비: 현재 landingX 를 맹목적으로 따라가지 않기

상대가 곧 공을 칠 상황에서는 현재 expectedLandingPointX 가 곧 무효가 됩니다. 그래서 상대가 공중에서 공 가까이에 있으면, 상대가 만들 수 있는 6 개 power-hit 계열을 현재 공 상태에서 가정해 착지점 묶음을 만들고 그 묶음을 가장 잘 커버하는 대기 x 를 찾습니다.

직관

테니스 리턴에서 상대 라켓에 공이 맞기 직전 “현재 공의 종착점”을 쫓는 게 아니라, 상대가 보낼 수 있는 방향들을 보고 가운데를 잡는 것과 비슷합니다.

다이빙은 마지막 수단

다이빙은 8px/frame 이라 걷기보다 빠르지만 이후 state 4 회복이 있습니다. PikaPlanner 는 일반 이동으로는 예상 낙하지점까지 reach 가 부족하고, dive 속도라면 살릴 수 있는 낮은 공에서만 발동하도록 제한합니다.

7. 코드 구조와 팀이 수정할 영역

영역	평소 수정?	대회 당일 스킴?	설명
[A] ENGINE CONSTANTS	거의 X	기본 물리가 바뀐 경우만	공개 물리 상수
[B] TUNABLE PARAMETERS	0	0	deadband, offset, horizon, 평가 가중치

[C] SKILL ADAPTER	평소 no-op	★ 1 순위	새 필드/행동/물리/resource 격리
[E] PHYSICS SIMULATOR	가급적 X	물리 변경 스킬이면 O	공/플레이어 rollout
[F] REACHABILITY	가급적 X	dash/teleport 면 O	도달 가능성 모델
[I] CORE PLANNER	가급적 X	최대한 X	접촉/수비/공격의 기본 구조
[J] ACTION SAFETY	검증 후 봉인	입력 의미가 바뀔 때만	stale 입력 사고 방지

파일 하나뿐이어도 “모듈 경계”를 주석과 함수 단위로 유지한 이유는 당일 수정 속도 때문입니다. 스킬이 나왔을 때 planner 전체를 읽으면서 조건문을 끼워 넣는 방식은 regression 위험이 큼니다.

8. 공개될 스킬에 대응하는 방법

스킬의 이름이나 연출보다 먼저 “무엇을 바꾸는 스킬인가”로 분류합니다. 아래 네 종류로 나누면 대부분 adapter 안에서 끝납니다.

분류	예시	주 수정 함수
새 행동	dash, shield, special hit	SKILL.extraActions(), emergencyOverride()
공/물리 변경	중력 변화, 공 가속/굴절	SKILL.simulateBallHook(), physics simulator
Reachability 변경	teleport, double jump, hitbox 확대	SKILL.transformReachability()
Resource	게이지, 쿨다운, 1 회성 사용	SKILL.read(), SKILL.evaluate()

스킬 공개 직후 30 분 동안 할 일

- 새 snapshot 필드 이름/타입/nullable 여부를 그대로 적는다.
- 발동 가능한 state, edge/hold trigger, collision 전/후 적용 순서를 찾는다.
- 가만히/이동/점프 상승/점프 하강/공 겹침 직전 등 단순 probe 를 반복한다.
- x/y/state 와 ball x/y/vx/vy 가 몇 frame 동안 어떻게 바뀌는지 표로 기록한다.
- adapter 와 simulator 를 먼저 수정하고, core planner 는 마지막까지 건드리지 않는다.
- 내 skill 만 보지 말고 상대 skill 이 상대 reachable set 을 얼마나 늘리는지도 같은 함수에 반영한다.

대회 당일 금지에 가까운 행동

스킬이 나왔다고 LEFT/RIGHT 변환, 기본 collision, action safety 까지 한꺼번에 뜯어고치지 마세요. 새 메커니즘을 adapter 에서 표현할 수 없는지 먼저 확인하는 것이 안전합니다.

9. 실험·튜닝 도구 사용법

9.1 빠른 회귀 테스트

```
cd pikachu_bot_package
node tools/verify.mjs
node tools/bench.mjs
```

verify 는 잘못된 반환값/예외를 먼저 잡고, 두 baseline 과 짧게 붙습니다. bench 는 decide() 자체가 갑자기 무거워졌는지 보는 regression 도구입니다.

9.2 여러 세트 자동 대전

```
node tools/run_series.mjs \
  --left ./PikaPlanner_tuned_v1.js \
```

```
--right ./tools/BaselinePositioning_v1.js \
--matches 50 --seed 100 --latency 1,2,3
```

반드시 LEFT/RIGHT 를 바꿔 한 번 더 돌리세요. 공개 엔진 주석 자체가 잔여 side bias 를 보고합니다. 또한 latency=1 만 잘 되는 버전보다 1/2/3 모두에서 버티는 버전을 우선합니다.

9.3 자동 파라미터 탐색

```
node tools/tune.mjs \
--bot ./PikaPlanner_v1.js \
--generations 8 --population 14 --matches 10 \
--seed 20260901 --out ./tools/best_config.json
```

```
node tools/export_tuned_bot.mjs \
--bot ./PikaPlanner_v1.js \
--config ./tools/best_config.json \
--out ./PikaPlanner_tuned_v2.js
```

튜너는 작은 evolutionary/random search 입니다. 목적은 “정답 파라미터”를 찾는 것이 아니라, 사람이 정한 planner 구조 안에서 손으로 맞추기 어려운 deadband/offset/가중치를 빨리 탐색하는 것입니다.

9.4 Power Hit probe

```
node tools/probe.mjs x=170 y=125 vy=11
```

접촉 지점과 직전 수직속도를 넣어 6 종 power hit 의 착지점/비행 frame 을 빠르게 비교합니다. 스킵이 공 물리를 바꾸면 가장 먼저 이 도구를 새 물리에 맞춰 고치는 것이 좋습니다.

10. 이번에 실제로 해본 오프라인 검증

제공한 simulator 에서 1/2/3 frame latency 를 섞어 현재 튜닝본을 두 단순 baseline 과 비교했습니다. 이 숫자는 대회 승률이 아니라 “코드 변경 뒤 망가지지 않았는지” 보는 내부 regression 지표입니다.

시험	결과
tuned bot LEFT vs 수비형 baseline, 30 세트	26 승 3 패 1 무
tuned bot RIGHT vs 수비형 baseline, 30 세트	25 승 4 패 1 무
tuned bot LEFT vs 공격형 baseline, 30 세트	22 승 8 패
tuned bot RIGHT vs 공격형 baseline, 30 세트	29 승 1 패
Node decide() 20,000 회	평균 약 0.062 ms/call, invalid action 0

왜 이 결과를 과신하면 안 되는가

자체 simulator 는 공개 physics 와 bot cadence 를 옮겼지만 브라우저 Worker scheduling 을 완전히 재현하지는 않으며, baseline 도 실제 참가자 분포가 아닙니다. 최종 제출 전에는 반드시 대회 저장소의 Chrome 화면에서 여러 버전을 직접 붙여야 합니다.

11. 팀 개발 루프: 가장 추천하는 방식

```
아이디어 하나만 변경
-> node tools/verify.mjs
-> latency 1/2/3 자동 대전
-> LEFT/RIGHT swap
-> 실제 Chrome bot-vs-bot
```

- > 결과/영상/실점 원인 기록
- > 괜찮으면 새 버전으로 보존
- > 다음 아이디어

한 번에 여러 휴리스틱을 바꾸면 승률이 올라도 이유를 모르게 됩니다. `MyTeam\_v11.js`, `v12.js` 처럼 버전을 나눠 "어떤 변경이 무엇을 개선했는지"를 남기는 것이 중요합니다.

추천 opponent 포트폴리오

- Positioning 류의 매우 안전한 수비형
- 네트 앞으로 빨리 붙는 공격형
- 항상 FAST DOWN 을 노리는 단순 공격형
- 로브/직선/하방을 섞는 랜덤형
- 이전 버전 우리 봇
- 실전에서 발견한 약점을 일부러 반복하는 exploit bot

튜너의 opponents 배열을 이 포트폴리오로 확장하면 특정 baseline 하나에 과적합되는 위험을 크게 줄일 수 있습니다.

12. 현재 버전의 한계와 제출 전 체크리스트

- 공개되지 않은 스킬은 아직 no-op adapter 상태입니다.
- player yVelocity/state-2 내부 delay/state-4 recovery 는 snapshot 에 없어서 일부를 추정합니다.
- 상대 reachability 는 full minimax 가 아니라 짧은 model-based heuristic 입니다.
- self-set touch 추적은 snapshot 변화로 추정하며, 5-touch 극한까지 활용하지 않습니다.
- 브라우저 scheduling 은 환경에 따라 달라질 수 있으므로 실제 Chrome 측정이 필요합니다.

제출 직전 체크

체크	완료
JS 파일명 규약 및 4MB 제한 확인	☐
LEFT/RIGHT 각각 20 세트 이상 실제 엔진 테스트	☐
기본 AI / 수비형 / 공격형 / self-play 테스트	☐
콘솔 예외 0 회	☐
의도치 않은 착지 직후 dive/re-jump 영상 확인	☐
4 분 시간 제한에서 앞설 때/뒤질 때 행동 확인	☐
스킬 adapter 반영 후 skill-off regression 확인	☐
최종 파일을 별도 버전으로 동결	☐

부록 A. 현재 핵심 파라미터를 어떻게 생각하면 되는가

파라미터	쉽게 말하면
MOVE_DEADBAND	목표 x 근처에서 좌우로 덜 흔들리게 하는 폭
STANDBY_X	상대 코트에 공이 있을 때 기본 대기 위치
BODY_RECEIVE_OFFSET	몸으로 받을 때 공보다 몇 px 뒤에 설치
JUMP_SEARCH_HORIZON	지금 점프했을 때 몇 frame 까지 접촉 가능성을 볼지
POWER_ARM_DISTANCE_X/Y	공중에서 power state 를 준비할 정도로 가까운 범위
DOWN_SMASH_MAX_NET_DISTANCE	하방 스매시를 아예 후보로 둘 최대 네트 거리
DIVE_MAX_LOOKAHEAD	얼마나 임박한 공까지 dive 구조를 검토할지
SHOT_* weights	상대 거리/시간 압박/도달 불가능성을 얼마나 중요하게 볼지

부록 B. 최종적으로 기억할 7 가지

1. 낙하지점은 보험이고, 강한 붓의 중심은 다음 contact 다.
2. 점프는 “높아서”가 아니라 미래 공 궤적과 점프 궤적이 겹칠 때 한다.
3. 일반 리시브도 접촉 geometry 를 설계한다.
4. 깊은 공은 한 번 self-set 해서 공격하기 좋은 타점을 만들 수 있다.
5. Power Hit 은 6 종 버튼 선택이 아니라 contact time 까지 포함한 궤적 최적화다.
6. 상대가 치기 직전에는 현재 landingX 보다 가능한 shot envelope 를 방어한다.
7. 스킬 당일에는 core planner 보다 adapter / physics / reachability / value 순서로 수정한다.